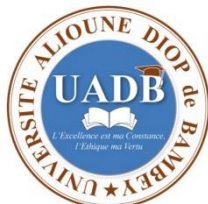


République du Sénégal



Un peuple - Un but - Une foi



Université Alioune Diop de Bambey

L'Excellence est ma constance, l'Éthique ma vertu

UFR Sciences Appliquées et Technologies de l'Information et de la Communication
(SATIC) Département Technologie de l'Information et de la Communication (TIC)

Master 1 Système d'informations

Projet Algorithme-Complexité

Parcours d'un Labyrinthe

Préparé par :

Aminata BA

Adja Fatou BASSE

Raky BOCOUM

Dr. Fatoumata BALDÉ KASSÉ

Année académique 2022-2023

Sommaire

Sommaire	2
Introduction.....	4
I. Notion de graphe.....	4
1. Graphes simples	4
2. Graphes orientés ou digraphes	6
3. Graphes pondérés	7
4. Graphes biparties	8
5. Graphes cycliques.....	9
6. Graphes acycliques orientés.....	10
7. Graphes pondérés orientés	11
8. Graphes connexes.....	12
9. Graphes réguliers.....	13
10. Arbres	14
II. Matrice d'adjacence, Matrice d'incidence et Liste d'adjacence.....	15
1. Matrice d'adjacence	15
2. Liste d'adjacence.....	17
3. Matrice d'incidence.....	18
III. Algorithmes et complexité sur les graphes	19
1. Définition	19
2. Algorithmes	19
a. DFS.....	19
b. BFS	20
3. Complexité.....	21
IV. Représentation du labyrinthe sous forme de graphe.....	21
1. Choix du langage de programmation	21
2. Avec le parcours en largeur	22
a. Principes	22
b. Démonstration.....	23
c. Calcul du plus court chemin avec DIJKSTRA	25
3. Avec le parcours en longueur	26
a. Principes	26

b. Démonstration.....	26
Conclusion	28
➤ Références	29

Introduction

Au cœur de tout défi, que ce soit dans un labyrinthe ou dans la résolution de problèmes informatiques complexes, se trouve souvent la nécessité de comprendre et de naviguer à travers des structures de données. Imaginez-vous perdu dans un labyrinthe, sans aucune indication sur la direction à suivre pour atteindre la sortie.

Dans une telle situation, représenter ce défi sous forme de graphe implicite peut être une approche pertinente. Chaque intersection du labyrinthe peut être vue comme un nœud du graphe, et chaque chemin entre deux intersections, une arête. Trouver un chemin reliant le nœud d'entrée au nœud de sortie devient alors l'objectif principal. Face à ce défi, adopter une stratégie de parcours en profondeur, explorant chaque chemin jusqu'à trouver la sortie, peut s'avérer efficace.

Les graphes sont des structures de données omniprésentes dans de nombreux domaines de l'informatique et des mathématiques. Leur flexibilité permet de modéliser une grande variété de situations, des réseaux sociaux à la logistique des transports. Dans ce contexte, il est crucial de comprendre les fondamentaux des graphes, y compris leurs types, leurs représentations et les algorithmes associés à leur manipulation.

I. Notion de graphe

Un graphe est défini par un ensemble de sommets S et un ensemble d'arêtes A qui sont des paires de sommets. Plus formellement, un graphe est un couple $G=(S, A)$ où S est un ensemble fini de sommets (aussi appelés nœuds) et $A \subseteq S \times S$ est un ensemble de couples de sommets définissant une relation binaire sur S . Les graphes peuvent être représentés de différentes manières, chacune ayant ses avantages et ses inconvénients en termes de complexité algorithmique et d'espace mémoire. Les deux représentations les plus courantes sont la matrice d'adjacence et les listes d'adjacence. La matrice d'adjacence est une matrice carrée où chaque ligne et chaque colonne correspondent à un nœud du graphe, et le coefficient à la i ème ligne et la j ème colonne est égal à 1 si il existe une arête entre les nœuds i et j , et 0 sinon. Les listes d'adjacence, d'autre part, sont des tableaux de listes, où chaque entrée de la table correspond à un nœud et chaque liste contient les nœuds adjacents. Cette partie sera mieux explicitée dans la suite.

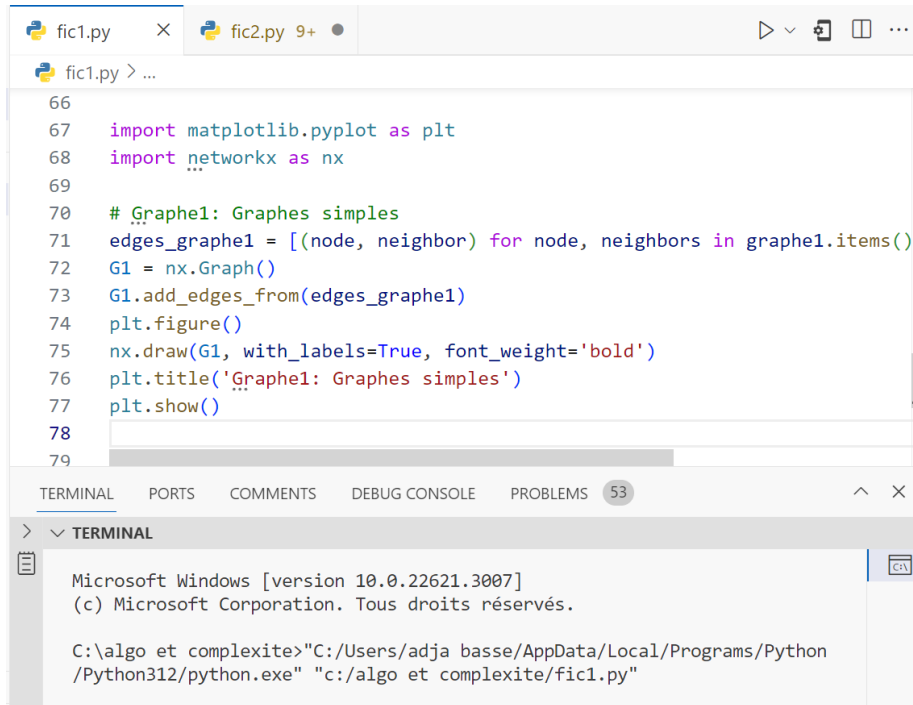
Le type de graphe se réfère aux caractéristiques structurelles et comportementales du graphe lui-même, tandis que la représentation du graphe se réfère à la manière dont le graphe est stocké en mémoire ou visualisé. Ils se déclinent en différents types, chacun avec ses caractéristiques spécifiques.

1. Graphes simples

Les graphes simples sont des graphes non orientés où chaque paire de nœuds est reliée par au plus une arête.

Graphe1 = {

```
'A': ['B', 'C'],  
'B': ['A', 'C'],  
'C': ['A', 'B'] }
```

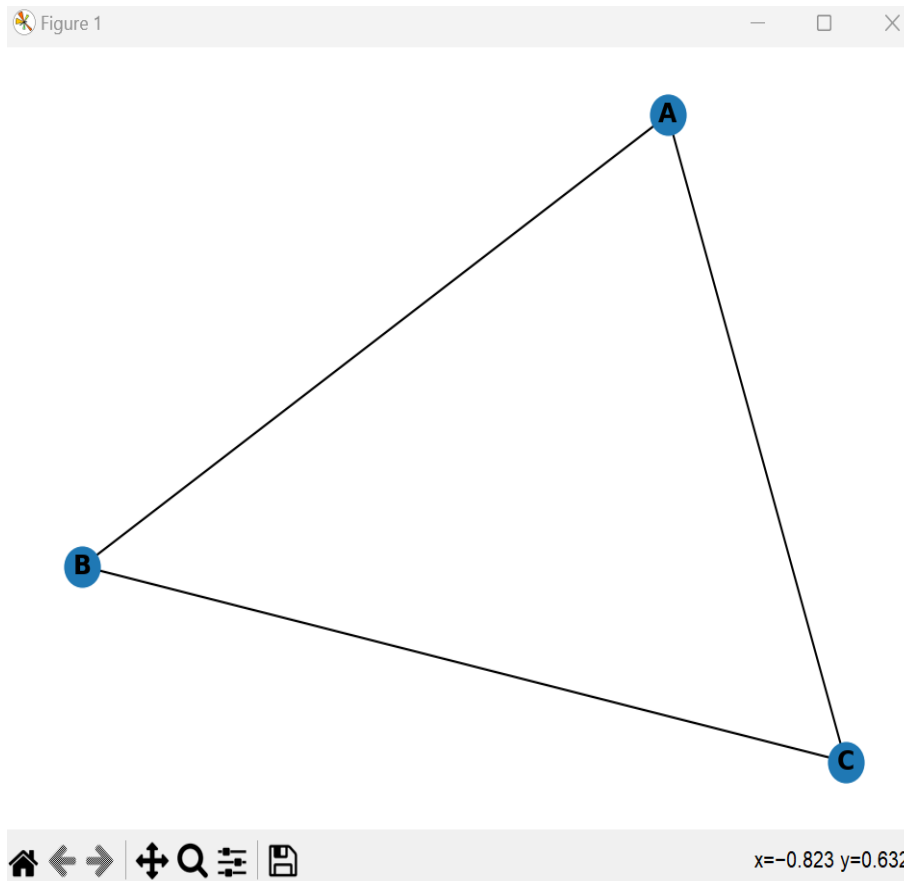


The image shows a code editor with two tabs: 'fic1.py' and 'fic2.py'. The 'fic1.py' tab is active, displaying the following Python code:

```
66  
67 import matplotlib.pyplot as plt  
68 import networkx as nx  
69  
70 # Graphe1: Graphes simples  
71 edges_graphe1 = [(node, neighbor) for node, neighbors in graphe1.items()  
72 G1 = nx.Graph()  
73 G1.add_edges_from(edges_graphe1)  
74 plt.figure()  
75 nx.draw(G1, with_labels=True, font_weight='bold')  
76 plt.title('Graphe1: Graphes simples')  
77 plt.show()  
78  
79
```

Below the code editor is a terminal window titled 'TERMINAL'. It shows the output of running the script:

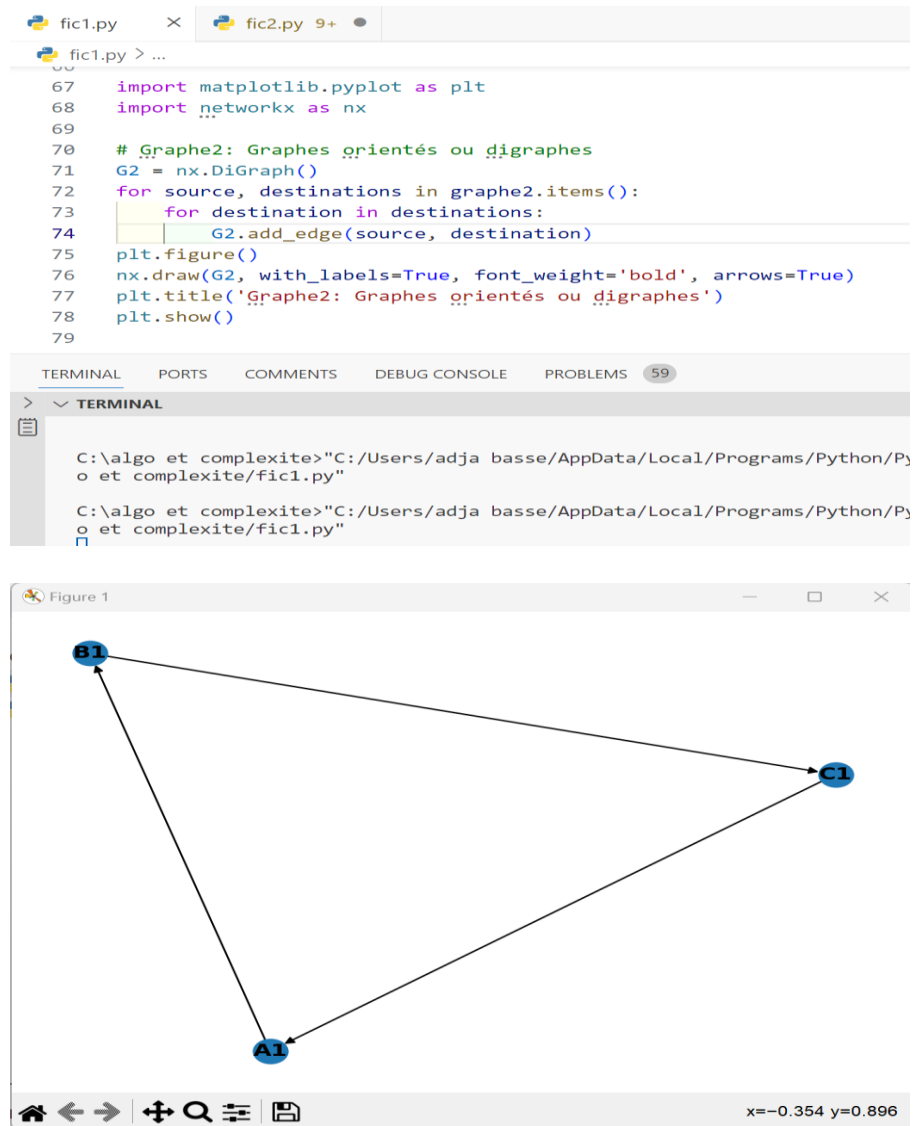
```
Microsoft Windows [version 10.0.22621.3007]  
(c) Microsoft Corporation. Tous droits réservés.  
  
C:\algo et complexite>"C:/Users/adja basse/AppData/Local/Programs/Python  
/Python312/python.exe" "c:/algo et complexite/fic1.py"
```



2. Graphes orientés ou digraphes

Les graphes orientés sont des graphes où chaque arête est orientée, c'est-à-dire qu'elle a une direction spécifique.

```
Graphe2 = {  
    'A1': ['B1'],  
    'B1': ['C1'],  
    'C1': ['A1'],  
    'D': [],  
}
```



3. Graphes pondérés

Les graphes pondérés sont des graphes où chaque arête a un poids associé, représentant une certaine valeur ou distance.

```
Graphe3= {
    'Adja': [('Raky', 23)],
    'Raky': [('Amina', 24)],
    'Amina': [('Adja', 25)]
}
```

```

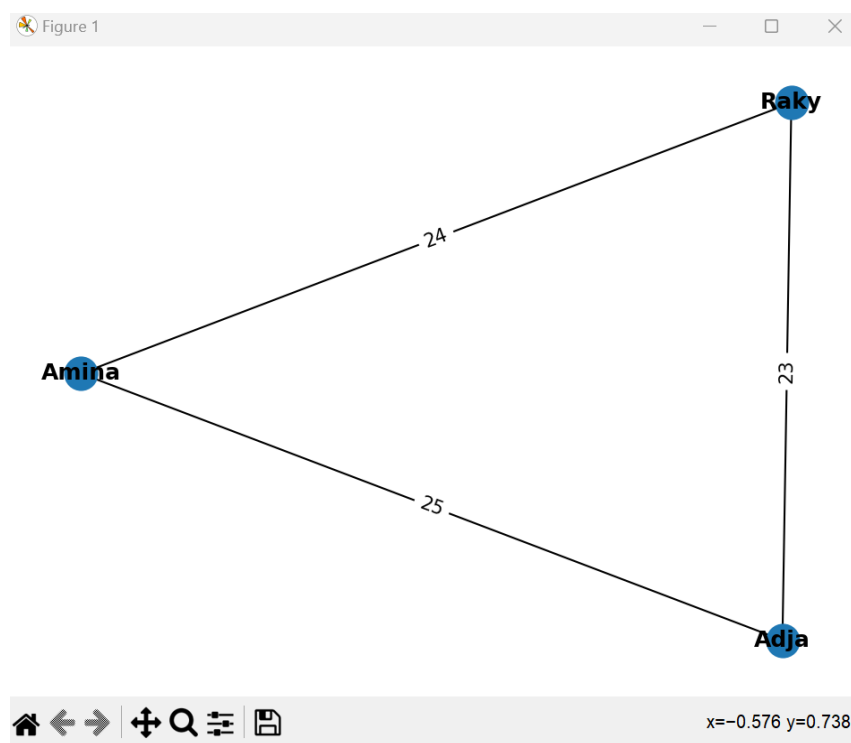
fic1.py x fic2.py 9+
fic1.py > ...
67 import matplotlib.pyplot as plt
68 import networkx as nx
69
70 # Graphe3: Graphes pondérés
71 G3 = nx.Graph()
72 for node, edges in graphe3.items():
73     for edge, weight in edges:
74         G3.add_edge(node, edge, weight=weight)
75 plt.figure()
76 pos = nx.spring_layout(G3)
77 nx.draw(G3, pos, with_labels=True, font_weight='bold')
78 nx.draw_networkx_edge_labels(G3, pos, edge_labels={(u, v): d['weight'] for u, v, d in G3.edges(data=True)})
79 plt.title('Graphe3: Graphes pondérés')
80 plt.show()
81

```

o et complexe/fic1.py

C:\algo et complexe>"C:/Users/adjja basse/AppData/Local/Programs/Python/Python312/python.exe" "c:/algo et complexe/fic1.py"

C:\algo et complexe>]



4. Graphes biparties

Les graphes bipartis sont des graphes dont l'ensemble de nœuds peut être divisé en deux ensembles distincts, et où chaque arête relie un nœud d'un ensemble à un nœud de l'autre ensemble.

Graphe4 = {

' Profs': ['Monsieur Basse', 'Madame Diop'],

'Élèves': ['Amina', 'Raky']

}

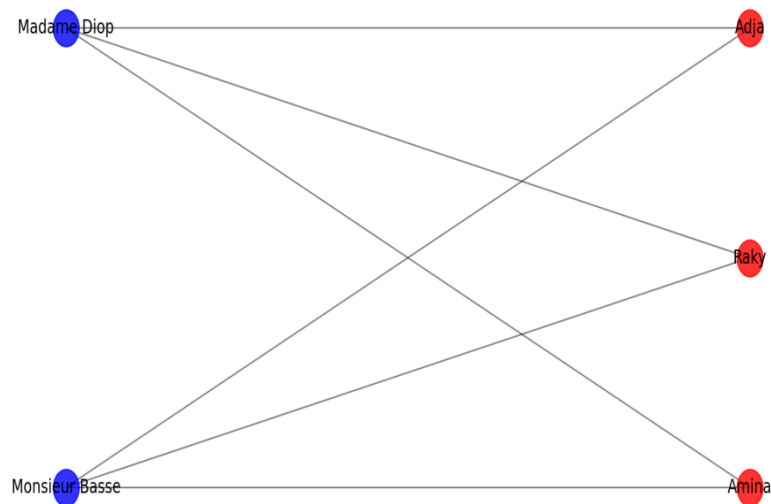
```
fic1.py x fic2.py 9+
fic1.py > ...
68 import networkx as nx
69
70 # Graphe4: Graphes bipartis
71 G = nx.Graph()
72 G.add_nodes_from(graphe4['Profs'], bipartite=0)
73 G.add_nodes_from(graphe4['Élèves'], bipartite=1)
74 for prof in graphe4['Profs']:
75     for eleve in graphe4['Élèves']:
76         G.add_edge(prof, eleve)
77 pos = nx.bipartite_layout(G, graphe4['Profs'])
78 nx.draw_networkx_nodes(G, pos, nodelist=graphe4['Profs'], node_color='b', node_size=500, alpha=0.8)
79 nx.draw_networkx_nodes(G, pos, nodelist=graphe4['Élèves'], node_color='r', node_size=500, alpha=0.8)
80 nx.draw_networkx_edges(G, pos, width=1.0, alpha=0.5)
81 nx.draw_networkx_labels(G, pos)
82 plt.title("Graphe biparti")
83 plt.axis('off')
84 plt.show()
```

TERMINAL PORTS COMMENTS DEBUG CONSOLE PROBLEMS 59

> TERMINAL

C:\algo et complexite>"C:/Users/adj a basse/AppData/Local/Programs/Python/Python312/python.exe" "c:/alg o et complexite/fic1.py"

Graphe biparti



5. Graphes cycliques

Les graphes cycliques sont des graphes qui contiennent au moins un cycle, c'est-à-dire un chemin fermé où le premier nœud est le même que le dernier nœud.

Graphe5 = {

'A1' : ['B1'],

'B1' : ['C1'],

'C1' : ['D1'],

'D1' : ['A1']

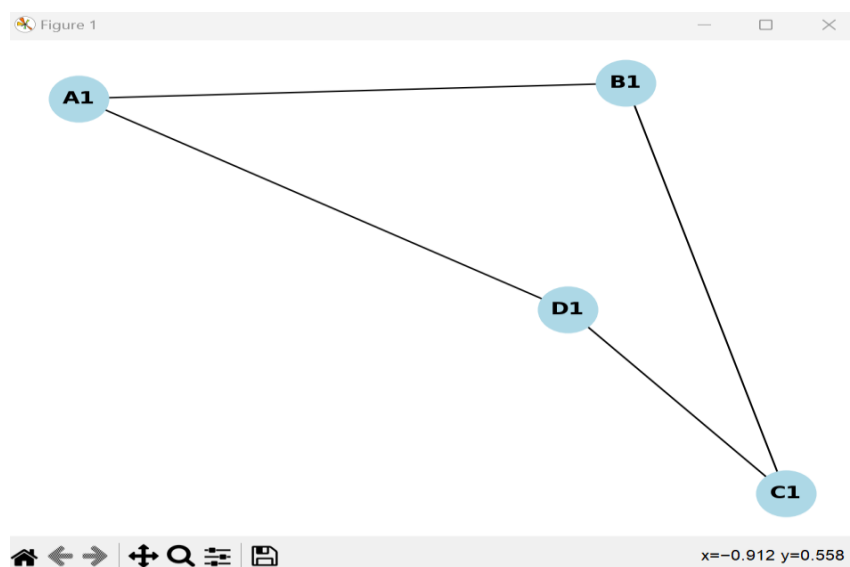
}

```
fic1.py x fic2.py 9+
fic1.py > ...
67 import matplotlib.pyplot as plt
68 import networkx as nx
69
70 # Graphe5: Graphes cycliques
71 G = nx.Graph()
72 G.add_edges_from([('A1', 'B1'), ('B1', 'C1'), ('C1', 'D1'), ('D1', 'A1')])
73 nx.draw(G, with_labels=True, node_color='lightblue', node_size=1000, font_size=12, font_weight='bold')
74 plt.title('Graphe Cyclique (graphe5)')
75 plt.show()
```

TERMINAL PORTS COMMENTS DEBUG CONSOLE PROBLEMS 68

> TERMINAL

C:\algo et complexite>"C:/Users/adja_basse/AppData/Local/Programs/Python/Python312/python.exe" "c:/alg
o et complexite/fic1.py"



6. Graphes acycliques orientés

Un graphe orienté qui ne contient pas de cycle.

Graphe6 = {

'A1': ['B1', 'C1'],

'B1': ['D1'],

'C1': ['D1'],

'D1': []

}

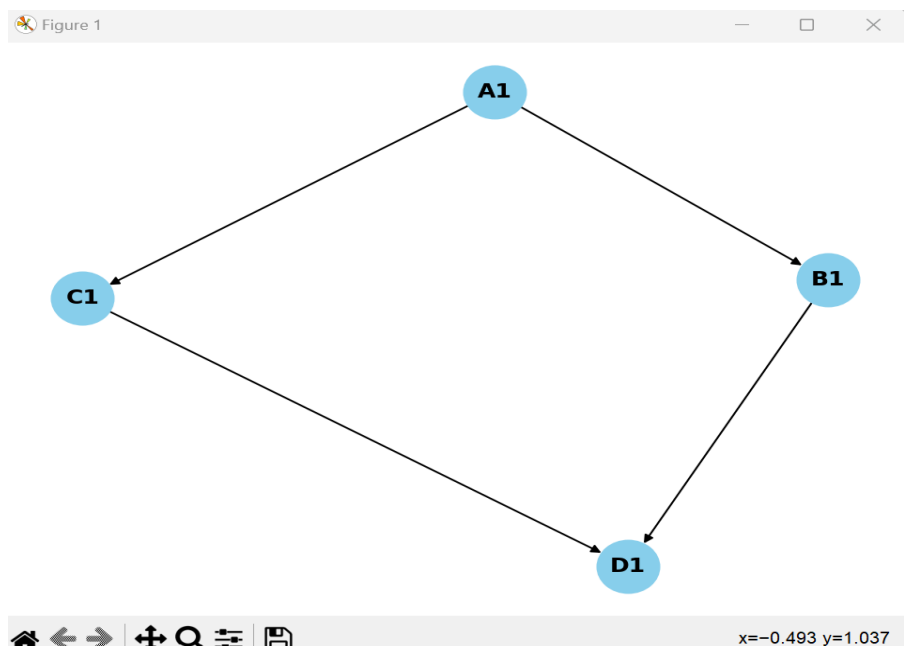
```
fic1.py X fic2.py 9+
fic1.py > ...
66
67 import matplotlib.pyplot as plt
68 import networkx as nx
69
70 # Graphe6: Graphe acyclique orienté
71 G = nx.DiGraph()
72 G.add_nodes_from(graphe6.keys())
73 for node, neighbors in graphe6.items():
74     for neighbor in neighbors:
75         G.add_edge(node, neighbor)
76 pos = nx.spring_layout(G)
77 nx.draw(G, pos, with_labels=True, node_size=1000, node_color="skyblue", font_size=12, font_weight="bold")
78 plt.title("Graphe acyclique orienté (graphe6)")
79 plt.show()
```

TERMINAL PORTS COMMENTS DEBUG CONSOLE PROBLEMS 66

> TERMINAL

C:\algo et complexite>"C:/Users/adja basse/AppData/Local/Programs/Python/Python312/python.exe" "c:/algo et complexite/fic1.py"

C:\algo et complexite>"C:/Users/adja basse/AppData/Local/Programs/Python/Python312/python.exe" "c:/algo et complexite/fic1.py"



7. Graphes pondérés orientés

Un graphe orienté où chaque arête a un poids associé.

Graphe7 = {

'A': [('B', 10)],

'B': [('C', 5)],

'C': [('D', 7)],

'D': []

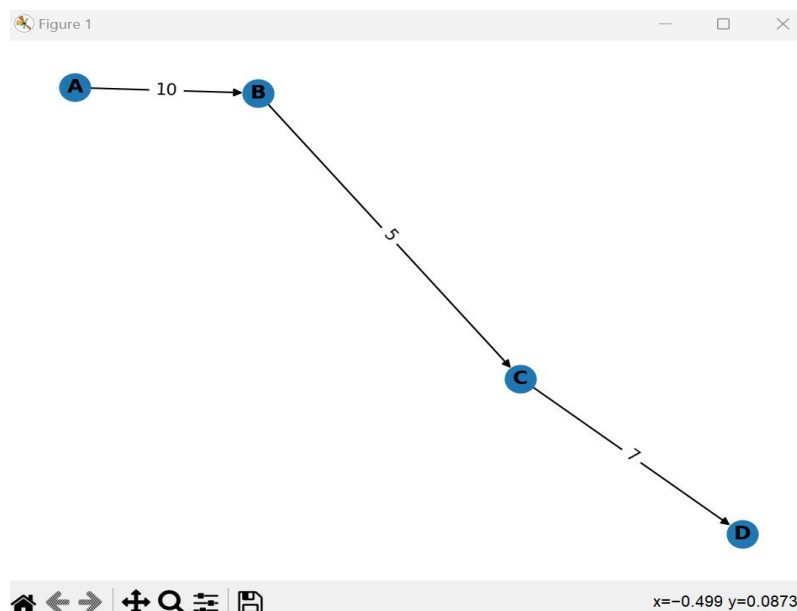
}

```
fic1.py x fic2.py 9+
fic1.py > ...
67 import matplotlib.pyplot as plt
68 import networkx as nx
69
70 # Graphe7: Graphe pondéré orienté
71 G7 = nx.DiGraph()
72 for node, edges in graphe7.items():
73     for edge, weight in edges:
74         G7.add_edge(node, edge, weight=weight)
75 plt.figure()
76 pos = nx.spring_layout(G7)
77 nx.draw(G7, pos, with_labels=True, font_weight='bold')
78 nx.draw_networkx_edge_labels(G7, pos, edge_labels={(u, v): d['weight'] for u, v, d in G7.edges(data=True)})
79 plt.title('Graphe7: Graphe pondéré orienté')
80 plt.show()
```

TERMINAL

```
C:\algo et complexite>"C:/Users/adja basse/AppData/Local/Programs/Python/Python312/python.exe" "c:/alg
o et complexite/fic1.py"

C:\algo et complexite>"C:/Users/adja basse/AppData/Local/Programs/Python/Python312/python.exe" "c:/alg
o et complexite/fic1.py"
```



8. Graphes connexes

Un graphe connexe est un graphe dans lequel il existe un chemin entre chaque paire de nœuds.

Graphe8 = {

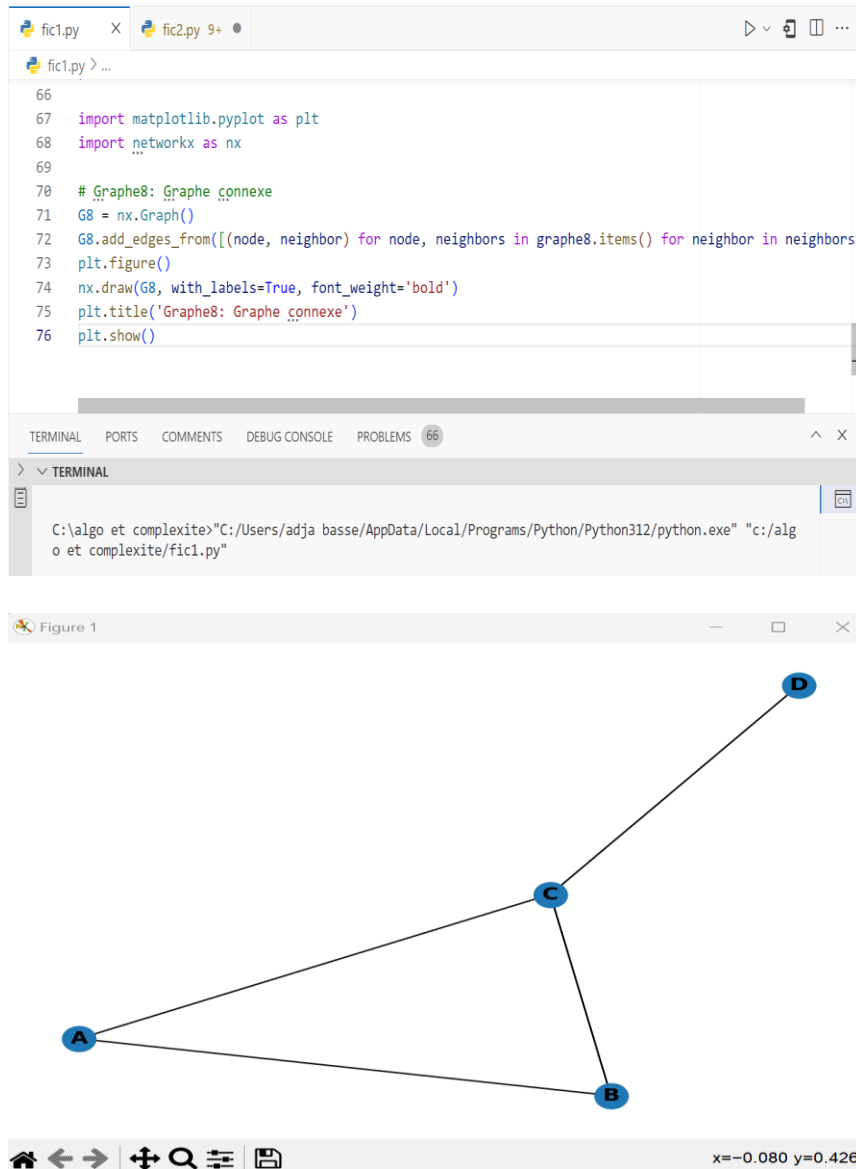
'A': ['B', 'C'],

'B': ['A', 'C'],

'C': ['A', 'B', 'D'],

'D': ['C']

}



9. Graphes réguliers

Un graphe régulier est un graphe dans lequel chaque nœud a le même degré, c'est-à-dire le même nombre d'arêtes adjacentes.

Graphe9 = {

'A': ['B', 'C'],

'B': ['A', 'C'],

'C': ['A', 'B']

}

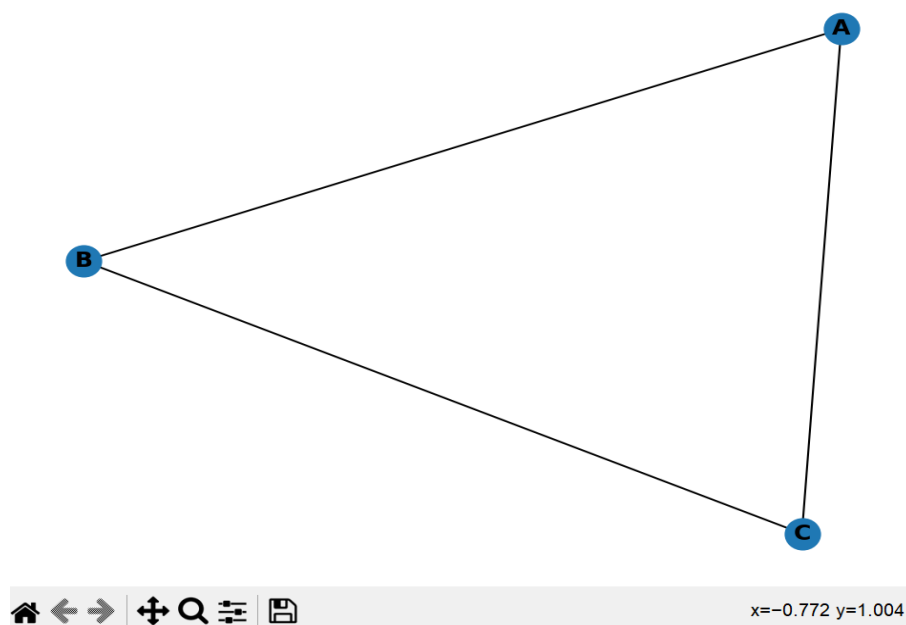
```
fic1.py x fic2.py 9+
fic1.py > ...
67 import matplotlib.pyplot as plt
68 import networkx as nx
69
70 # Graphe9: Graphes réguliers
71 G9 = nx.Graph()
72 G9.add_edges_from([(node, neighbor) for node, neighbors in graphe9.items() for neighbor in neighbors])
73 plt.figure()
74 nx.draw(G9, with_labels=True, font_weight='bold')
75 plt.title('Graphe9: Graphes réguliers')
76 plt.show()
77
```

TERMINAL PORTS COMMENTS DEBUG CONSOLE PROBLEMS 66

> TERMINAL

C:\algo et complexite>"C:/Users/adja basse/AppData/Local/Programs/Python/Python312/python.exe" "c:/alg
o et complexite/fic1.py"

Figure 1



10. Arbres

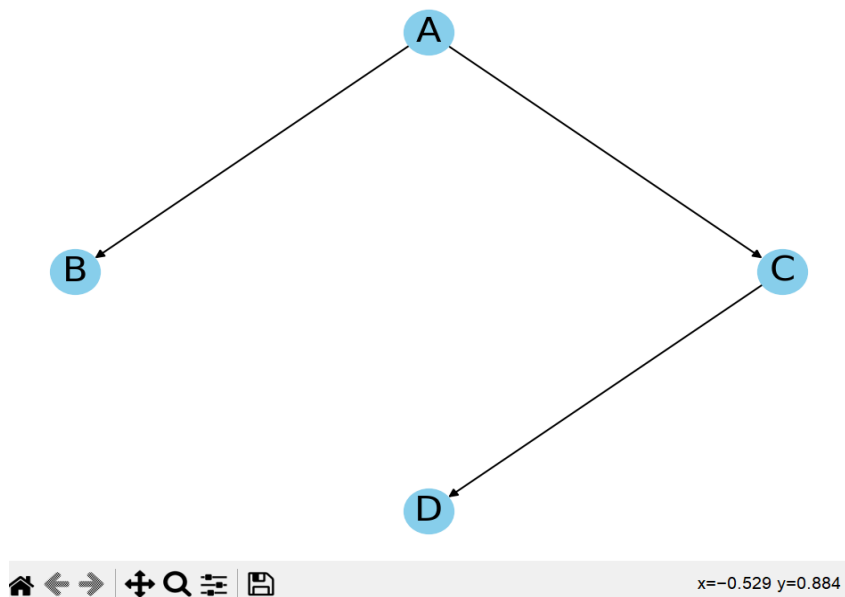
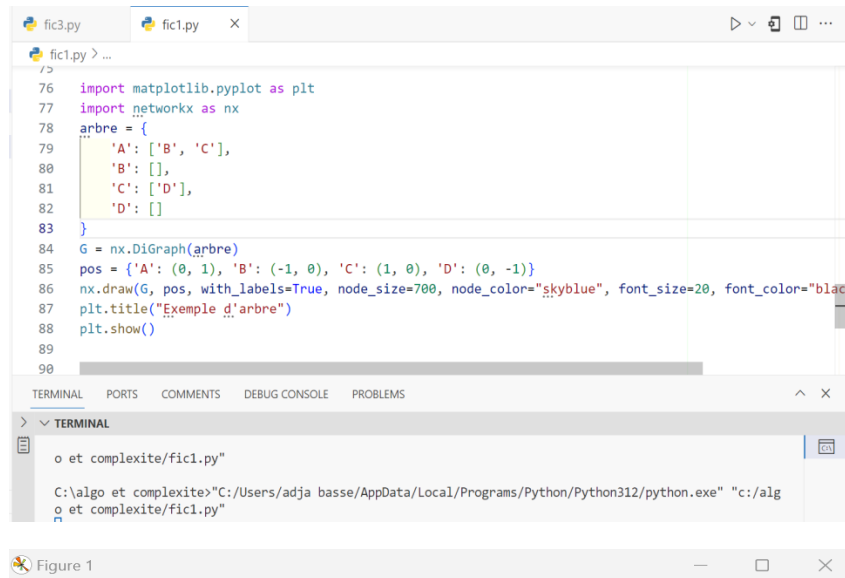
Ce sont des graphes, acycliques, connexes et non orientés.

arbre = {
'A': ['B', 'C'],
'B': [],

'C': ['D'],

'D': []

}

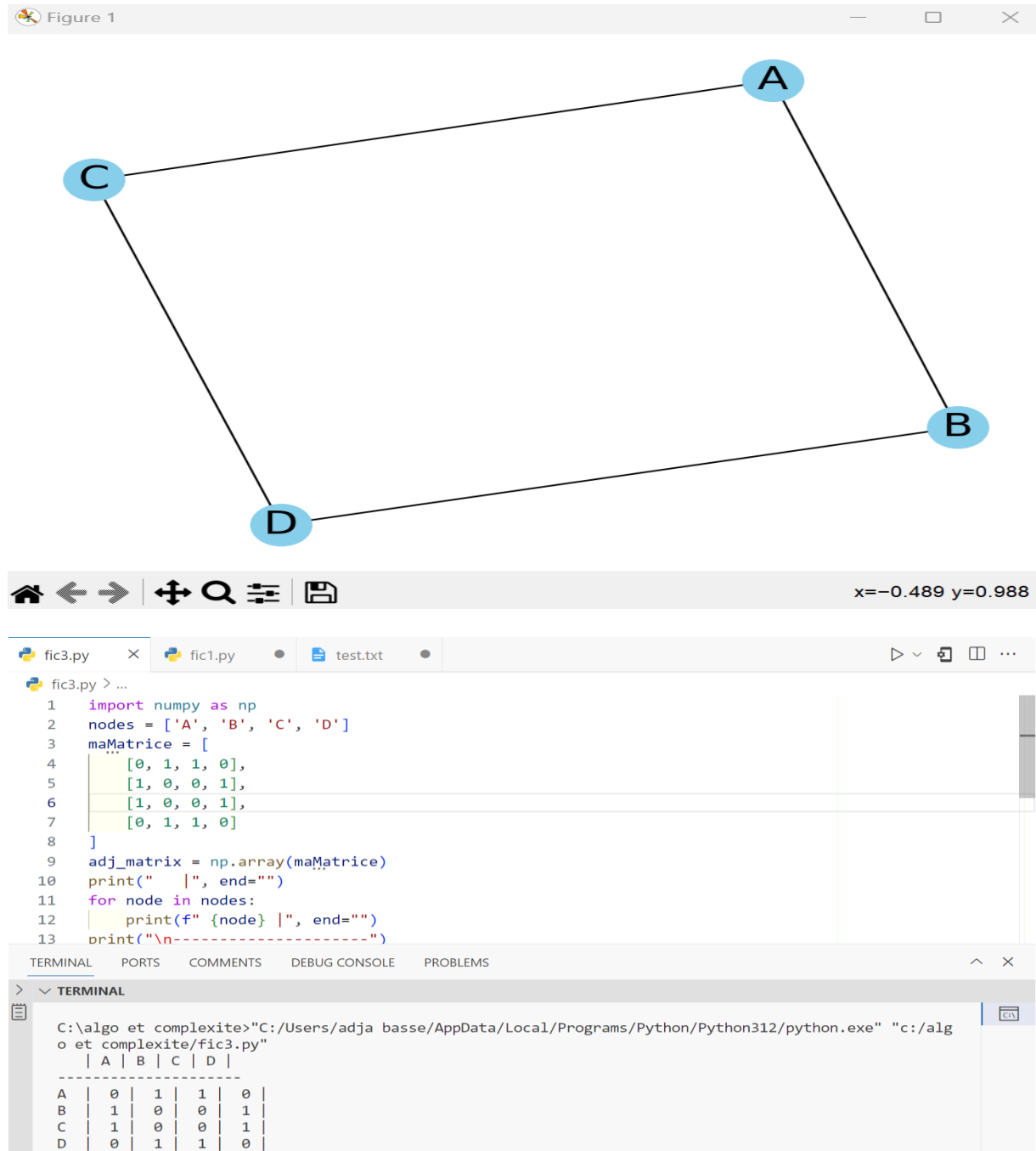


II. Matrice d'adjacence, Matrice d'incidence et Liste d'adjacence

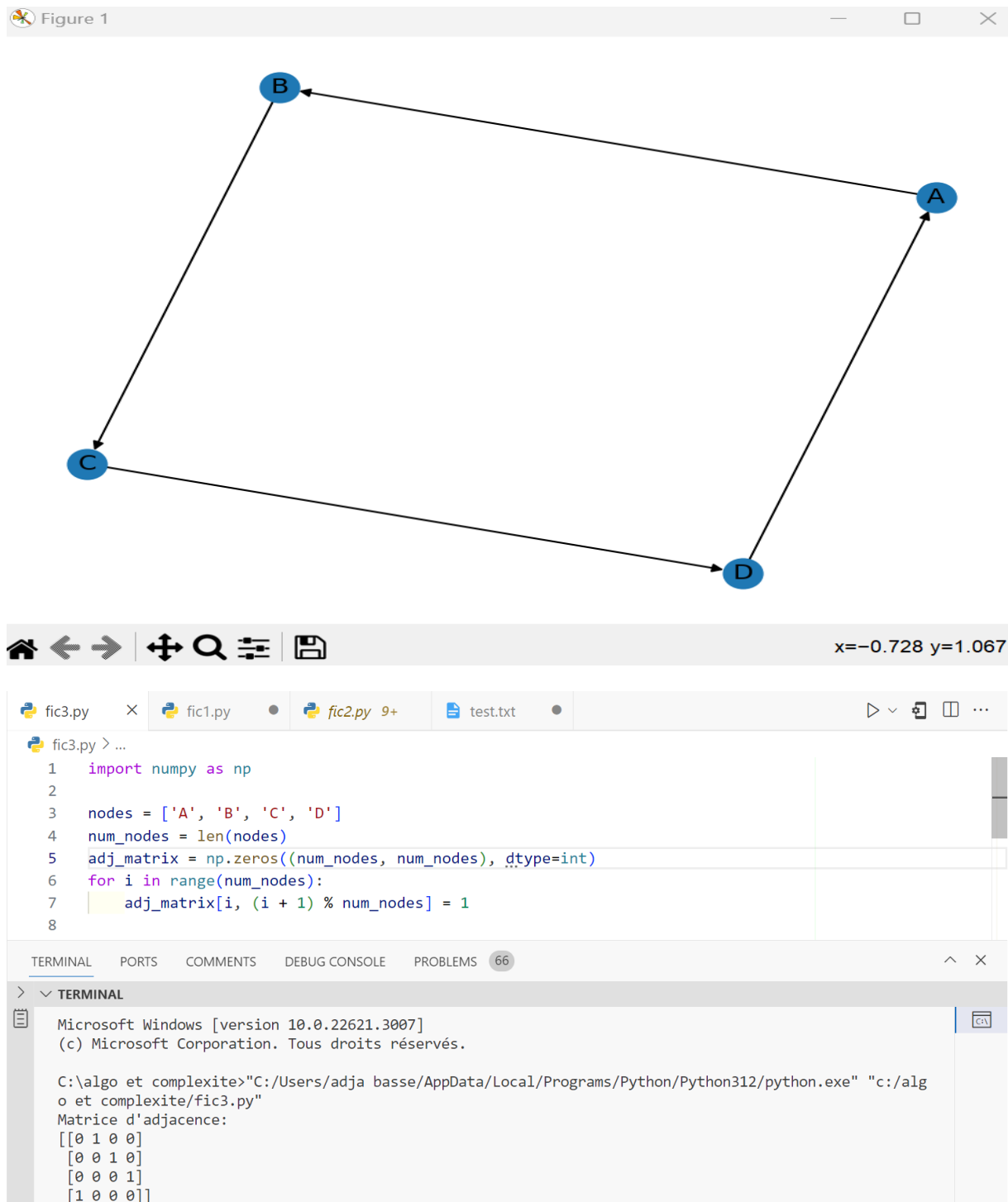
1. Matrice d'adjacence

Soit un graphe non orienté G avec n sommets. La matrice d'adjacence, notée M , est une matrice carrée de taille $n \times n$, où chaque ligne et chaque colonne représente un sommet du graphe. Si un sommet i est relié à un sommet j par une arête, alors $M[i][j]=1$, sinon $M[i][j]=0$

Graphe non-orientée :



Graphe orientée :



2. Liste d'adjacence

La liste d'adjacence est une représentation basée sur une structure de données linéaire qui stocke les voisins de chaque sommet dans une liste. Pour chaque sommet i , on crée une liste contenant tous les sommets adjacents à i . Ainsi, la liste d'adjacence est une liste de listes, où chaque élément de la liste extérieure correspond à un sommet, et chaque liste intérieure contient les voisins de ce sommet.

```
fic3.py x fic1.py fic2.py 9+ test.txt
fic3.py > ...
1 nodes = ['A', 'B', 'C', 'D']
2 edges = [(nodes[i], nodes[(i+1) % len(nodes)]) for i in range(len(nodes))]
3 adjacency_list = {node: [] for node in nodes}
4 for start, end in edges:
5     adjacency_list[start].append(end)
6     adjacency_list[end].append(start)
7 for node, neighbors in adjacency_list.items():
8     print(f"{node}: {neighbors}")

TERMINAL PORTS COMMENTS DEBUG CONSOLE PROBLEMS 66
> v TERMINAL
Microsoft Windows [version 10.0.22621.3007]
(c) Microsoft Corporation. Tous droits réservés.

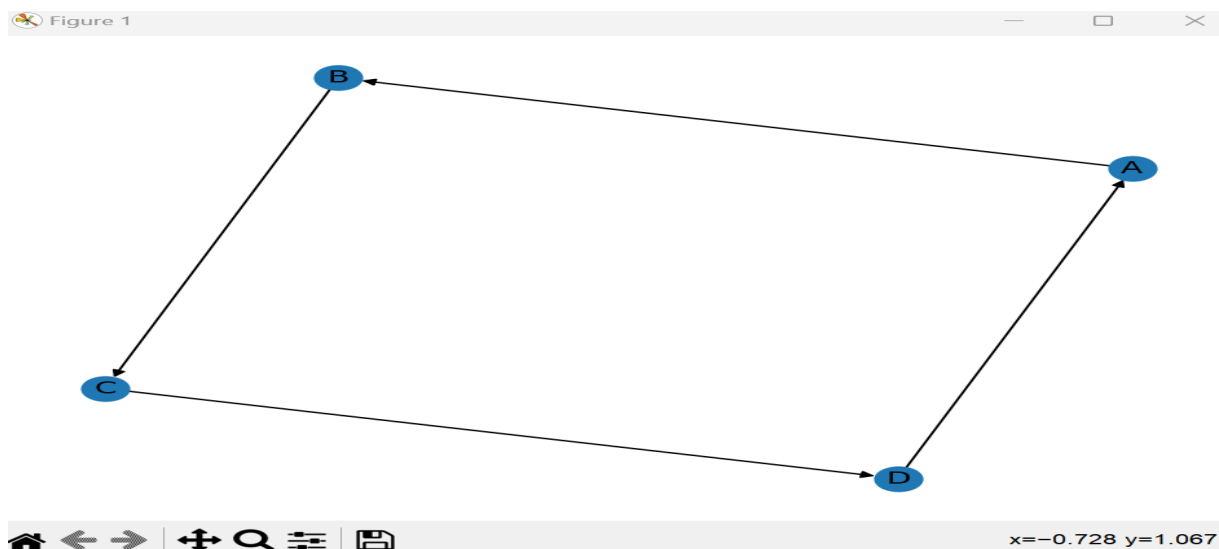
C:\algo et complexite>"C:/Users/adja basse/AppData/Local/Programs/Python/Python312/python.exe" "c:/alg
o et complexite/fic3.py"
A: ['B', 'D']
B: ['A', 'C']
C: ['B', 'D']
D: ['C', 'A']

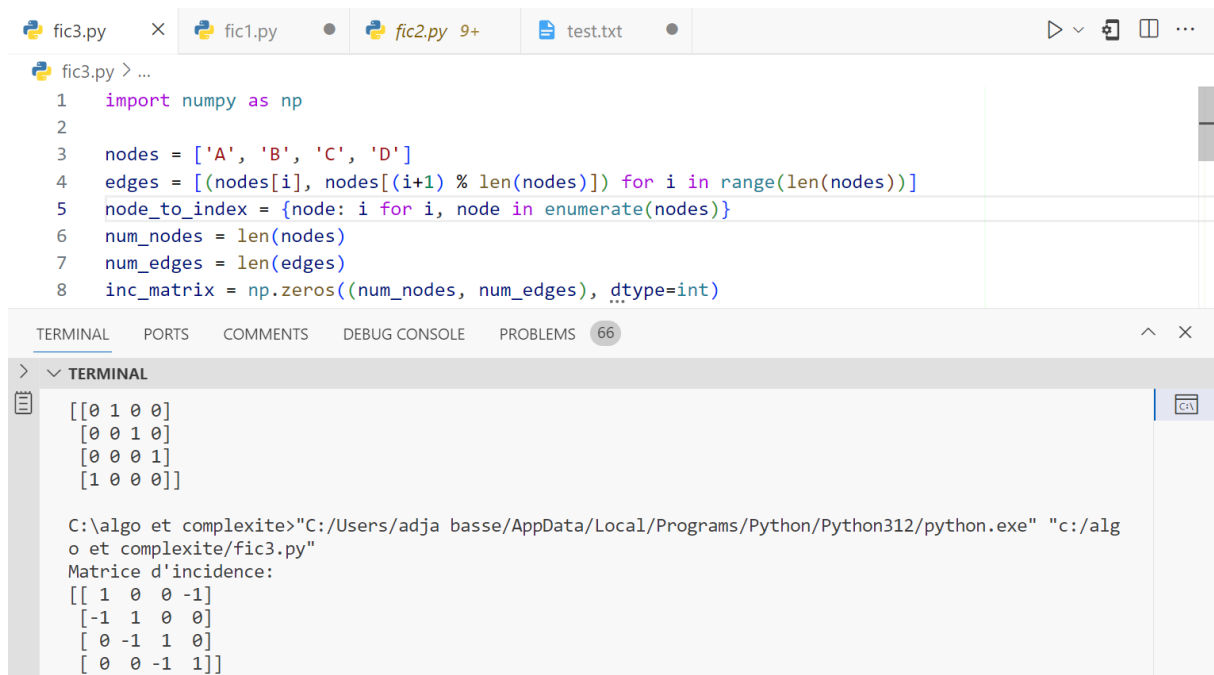
C:\algo et complexite>
```

3. Matrice d'incidence

La matrice d'adjacence indique simplement si deux sommets sont reliés par une arête, tandis que la matrice d'incidence indique quels sommets sont reliés par chaque arête spécifique.

C'est une matrice qui représente les relations entre les sommets et les arêtes du graphe. Chaque ligne de la matrice correspond à un sommet, chaque colonne correspond à une arête, et une entrée de la matrice est 1 si le sommet est lié à l'arête, sinon c'est 0.





The screenshot shows a Python IDE with a file explorer at the top containing 'fic3.py', 'fic1.py', 'fic2.py 9+', and 'test.txt'. The main editor displays the following Python code in 'fic3.py':

```
1 import numpy as np
2
3 nodes = ['A', 'B', 'C', 'D']
4 edges = [(nodes[i], nodes[(i+1) % len(nodes)]) for i in range(len(nodes))]
5 node_to_index = {node: i for i, node in enumerate(nodes)}
6 num_nodes = len(nodes)
7 num_edges = len(edges)
8 inc_matrix = np.zeros((num_nodes, num_edges), dtype=int)
```

Below the editor is a terminal window titled 'TERMINAL' showing the execution of the script:

```
> v TERMINAL
[[0 1 0 0]
 [0 0 1 0]
 [0 0 0 1]
 [1 0 0 0]]

C:\algo et complexite>"C:/Users/adja basse/AppData/Local/Programs/Python/Python312/python.exe" "c:/alg
o et complexite/fic3.py"
Matrice d'incidence:
[[ 1  0  0 -1]
 [-1  1  0  0]
 [ 0 -1  1  0]
 [ 0  0 -1  1]]
```

III. Algorithmes et complexité sur les graphes

1. Définition

Le parcours en profondeur (DFS - Depth First Search) et le parcours en largeur (BFS - Breadth First Search) sont deux algorithmes utilisés pour explorer des graphes. Le DFS explore autant que possible le graphe en suivant chaque branche jusqu'à ce qu'il atteigne le maximum de profondeur, puis revient en arrière pour explorer d'autres branches. En revanche, le BFS explore les nœuds voisins d'abord, en se déplaçant horizontalement à travers le graphe.

2. Algorithmes

a. DFS

Le parcours en profondeur (DFS) peut être implémenté de manière récursive ou itérative à l'aide d'une pile. En utilisant la récursivité, le DFS explore chaque branche jusqu'à atteindre le maximum de profondeur avant de revenir en arrière pour explorer les autres branches. En utilisant une pile, le DFS empile les nœuds à explorer et continue d'explorer les nœuds adjacents jusqu'à ce qu'il n'y ait plus de nœuds à explorer.

```
PP(G)
```

```
Pour chaque sommet  $u \in G.S$ 
```

```
u.couleur = BLANC
```

```
u.n = NIL
```

```
date = 0
```

```
Pour chaque sommet  $u \in G.S$ 
```

```
Si u.couleur = BLANC
```

```
Visiter_PP(G,u)
```

```
Visiter_PP(G,u)
```

```
date = date+1 //sommet blanc trouve
```

```
u.d=date
```

```
u.couleur = GRIS
```

```
Pour chaque  $v \in G.Adj[u]$  //explore arc(u,v)
```

```
si v.couleur == BLANC
```

```
v.n = u
```

```
Visiter_PP(G,v)
```

```
u.Couleur = NOIR
```

```
date = date+1
```

```
u.f=date
```

b. BFS

Le parcours en largeur (BFS) utilise une file d'attente pour explorer les nœuds en commençant par le nœud source et en se déplaçant horizontalement à travers le graphe. Il explore d'abord tous les nœuds à une distance de 1 du nœud source, puis tous les nœuds à une distance de 2, et ainsi de suite.

```
PL(G,s)
```

```
Pour chaque sommet  $u \in G.S - \{s\}$ 
```

```
u.couleur = BLANC
```

```
u.d =  $\infty$ 
```

```
u.n = NIL
```

```
s.couleur = GRIS
```

```
s.d = 0
```

```
s.n = NIL
```

```
F =  $\emptyset$ 
```

```
Enfiler(F,s)
```

```
Tant que  $F \neq \emptyset$ 
```

```
U = Defiler(F)
```

```
Pour chaque  $v \in G.Adj[u]$ 
```

```
si v.couleur == BLANC
```

```
v.couleur = GRIS
```

```
v.d = u.d+1
```

```
v.n = u
```

```
Enfiler(F,v)
```

```
u.couleur = NOIR
```

3. Complexité

La complexité des algorithmes sur les graphes est souvent évaluée en fonction du nombre de nœuds (N) et d'arêtes (E) dans le graphe. Cette évaluation est cruciale pour comprendre les performances des algorithmes en fonction de la taille du graphe. L'expression de la complexité sous la forme de $O(N + E)$ indique que le temps d'exécution de l'algorithme dépend linéairement du nombre de nœuds et d'arêtes dans le graphe. Cela signifie que le temps nécessaire pour exécuter l'algorithme augmente de manière linéaire avec le nombre de nœuds et d'arêtes. De même, l'espace mémoire requis pour stocker les données intermédiaires pendant l'exécution de l'algorithme est souvent proportionnel au nombre de nœuds dans le graphe, ce qui donne une complexité d'espace de $O(N)$. Ainsi, en analysant la complexité des algorithmes sur les graphes, les programmeurs peuvent estimer les performances et les limitations de leurs solutions algorithmiques en fonction de la taille des données d'entrée.

Temps : $O(N + E)$

Espace : $O(N+E)$

IV. Représentation du labyrinthe sous forme de graphe

1. Choix du langage de programmation

Python est un langage de programmation haut niveau, ce qui signifie qu'il est facile à lire et à écrire. Il dispose d'une vaste bibliothèque standard qui inclut des structures de données et des fonctionnalités essentielles pour la manipulation de graphes, telles que des listes, des dictionnaires et des ensembles, ainsi que des modules spécifiques aux graphes comme NetworkX, Matplotlib et Pygame.



2. Avec le parcours en largeur

a. Principes

Le principe du BFS dans un labyrinthe consiste à explorer les cases les plus proches de la position de départ en premier, puis à se déplacer progressivement vers les cases situées à une distance plus grande. On utilise une file d'attente pour stocker les cases à explorer, en commençant par la case de départ. À chaque étape, on explore les cases voisines de la case en tête de file, puis on les ajoute à la file d'attente pour les explorer ultérieurement. Ce processus se poursuit jusqu'à ce que la sortie soit trouvée ou que toutes les cases accessibles aient été explorées.

b. Démonstration

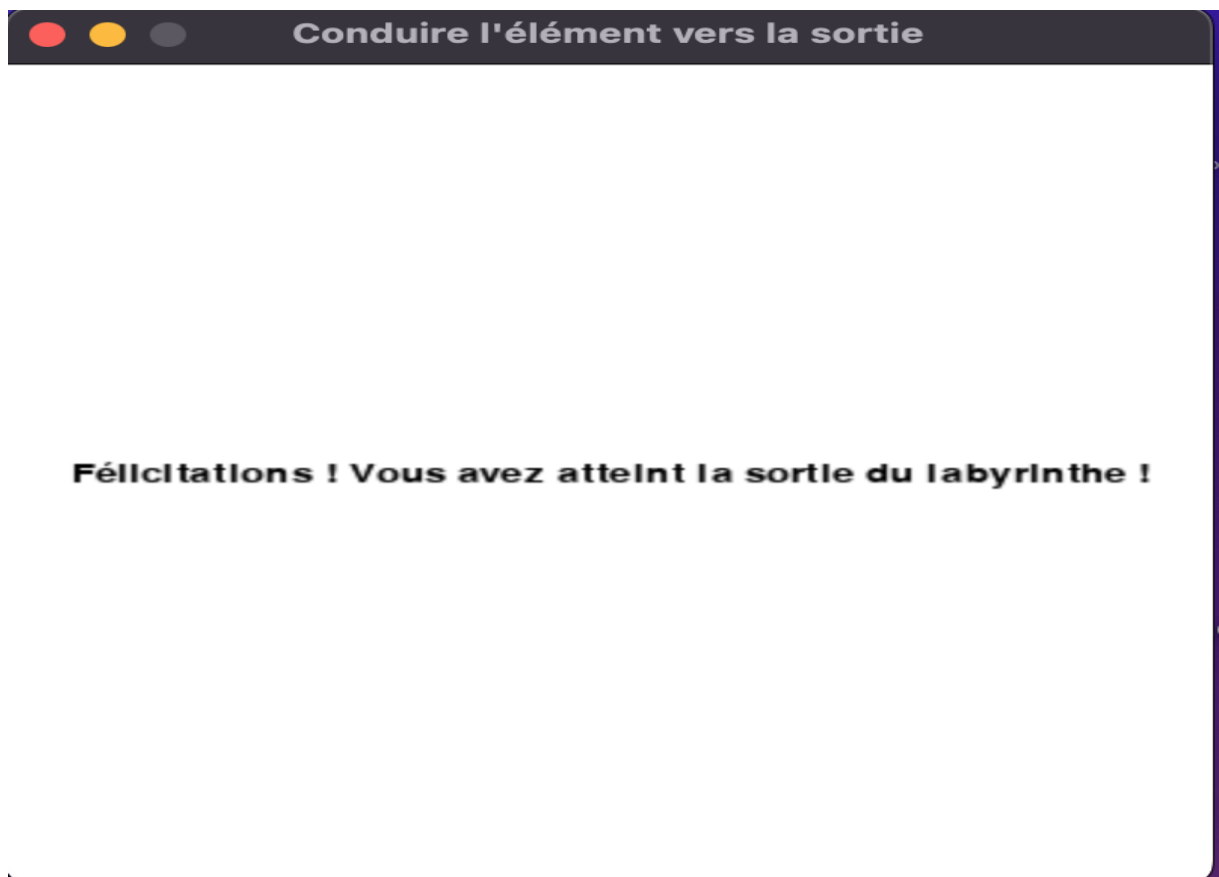


Si le temps est épuisé :





Vous avez réussi !



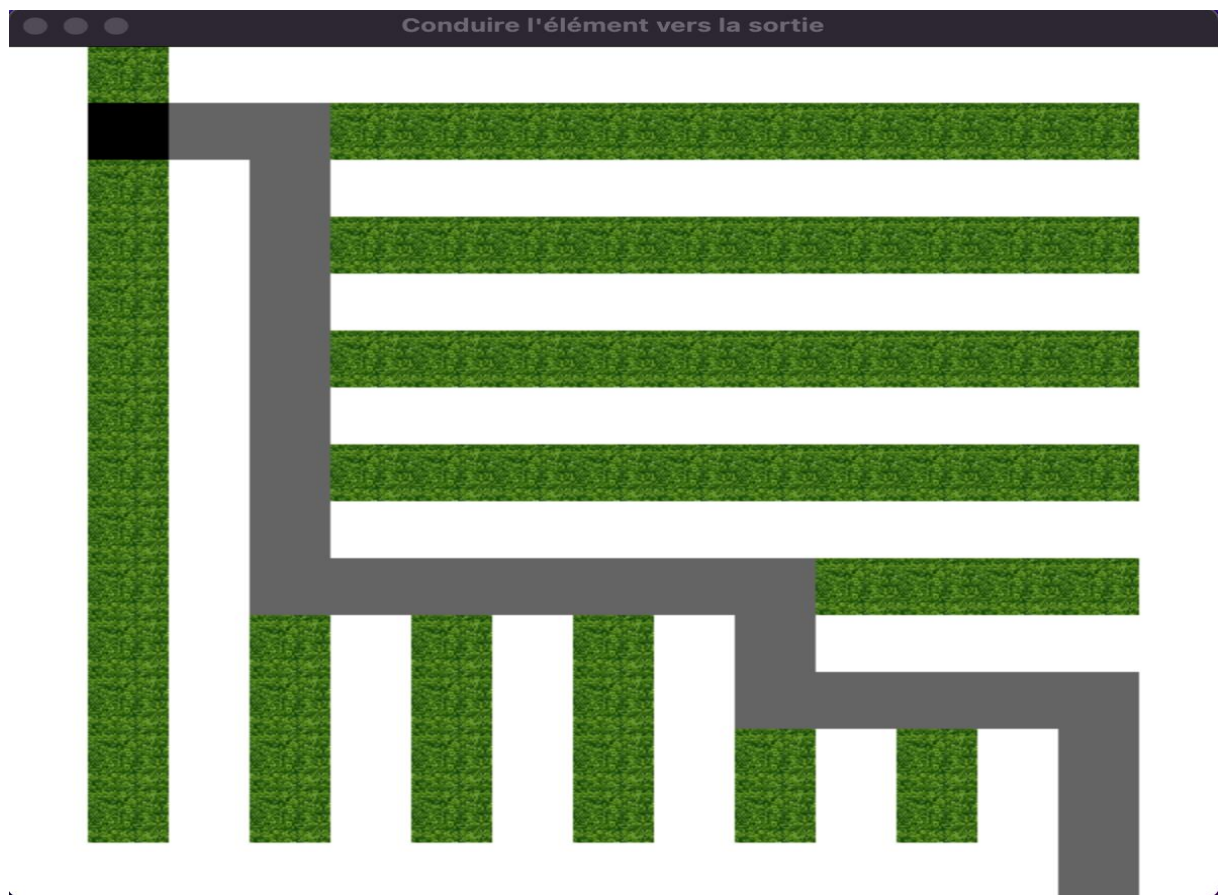
c. Calcul du plus court chemin avec DIJKSTRA

L'algorithme de Dijkstra est utilisé pour trouver le chemin le plus court entre un nœud source et tous les autres nœuds dans un graphe pondéré et orienté ou non orienté. L'algorithme fonctionne en attribuant une valeur de distance initiale à chaque nœud, puis en mettant à jour ces valeurs en fonction des chemins les plus courts trouvés jusqu'à présent. Il explore les nœuds de manière itérative en choisissant toujours le nœud non visité avec la distance minimale par rapport au nœud source.

Dans le contexte du parcours en largeur (BFS), l'algorithme de Dijkstra peut être adapté pour calculer le plus court chemin en parcourant le labyrinthe de manière systématique en se déplaçant horizontalement d'abord.

Pour ce faire, à chaque étape du BFS, au lieu d'explorer simplement les cases voisines, on calcule également la distance entre la case actuelle et ses voisins, et on met à jour la distance minimale pour chaque voisin en fonction des chemins les plus courts trouvés jusqu'à présent.

On continue à explorer le labyrinthe en utilisant le BFS, tout en mettant à jour les distances minimales à chaque étape, jusqu'à ce que la sortie soit trouvée ou que toutes les cases accessibles aient été explorées.

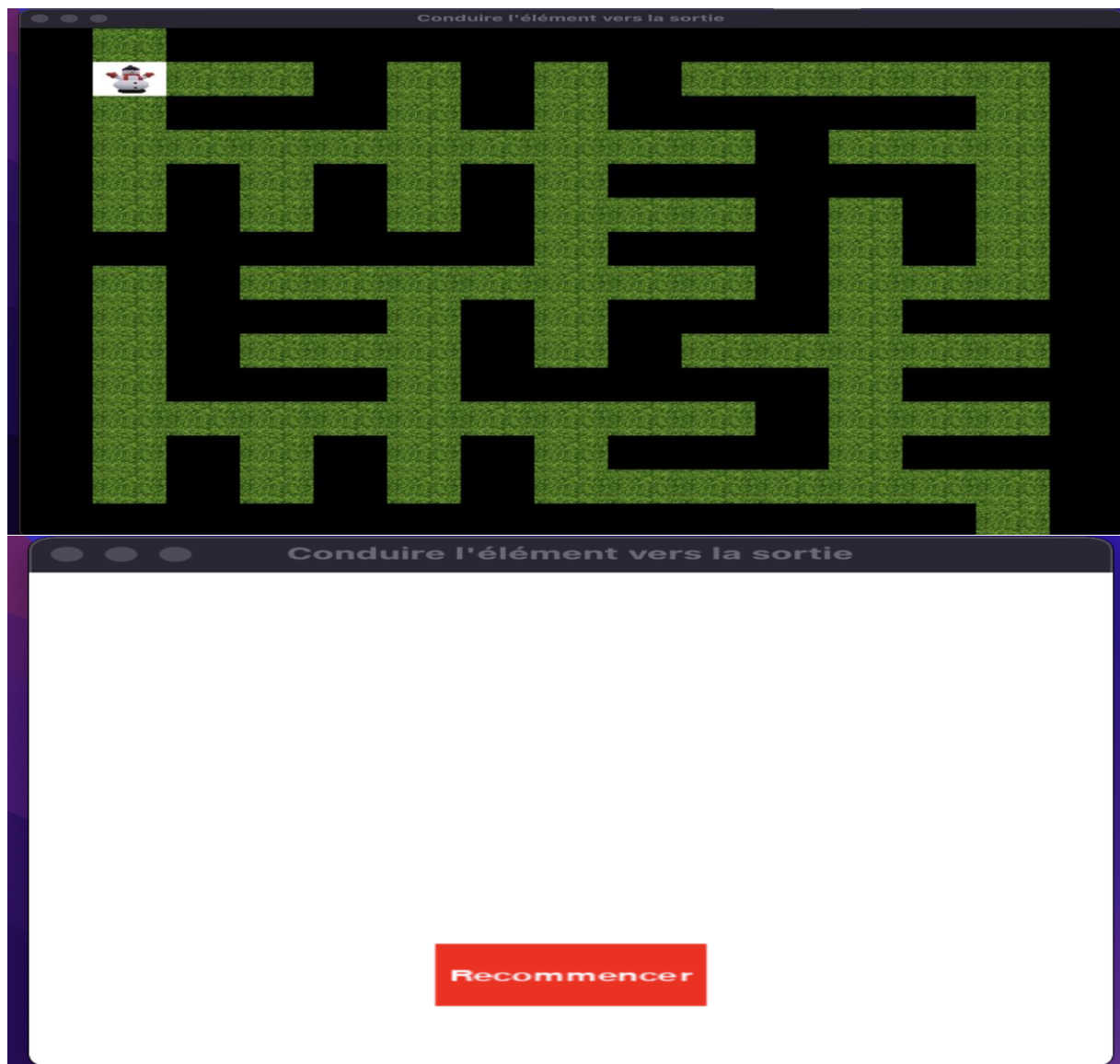


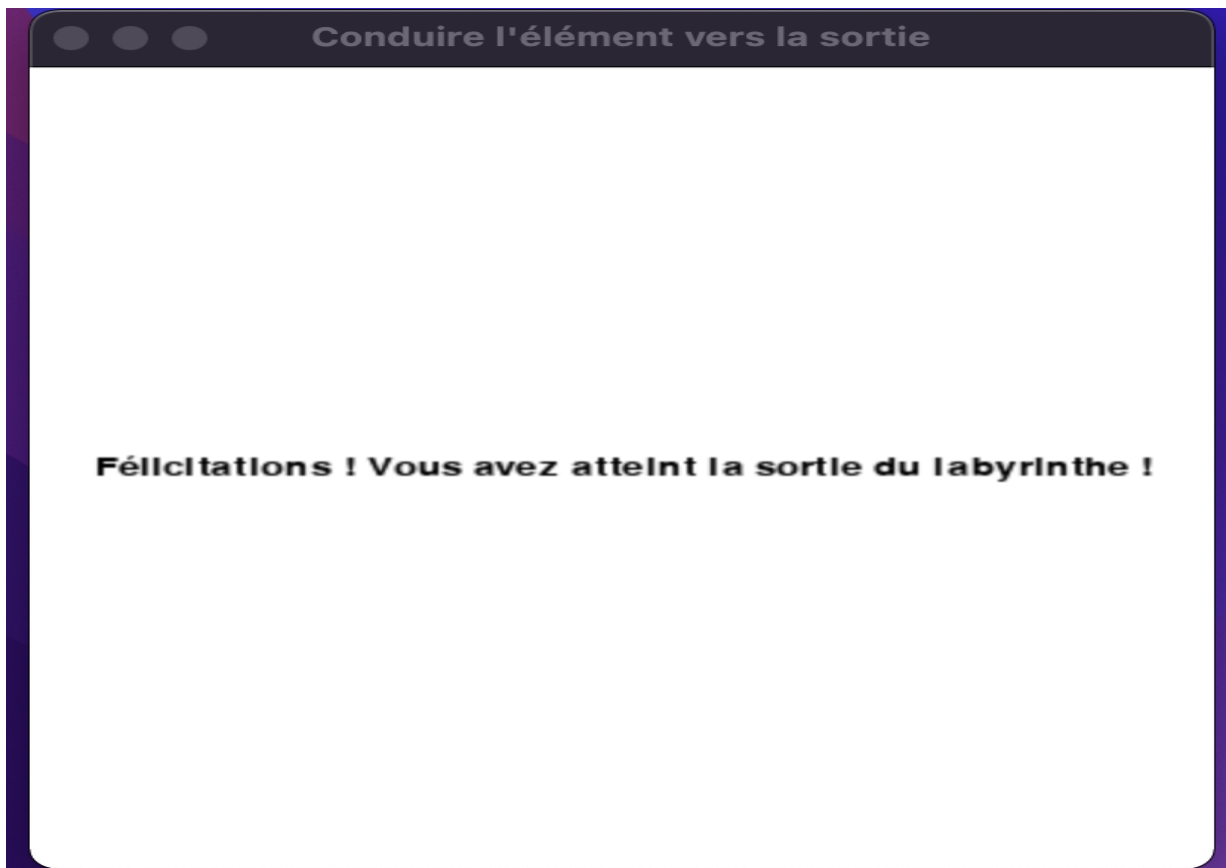
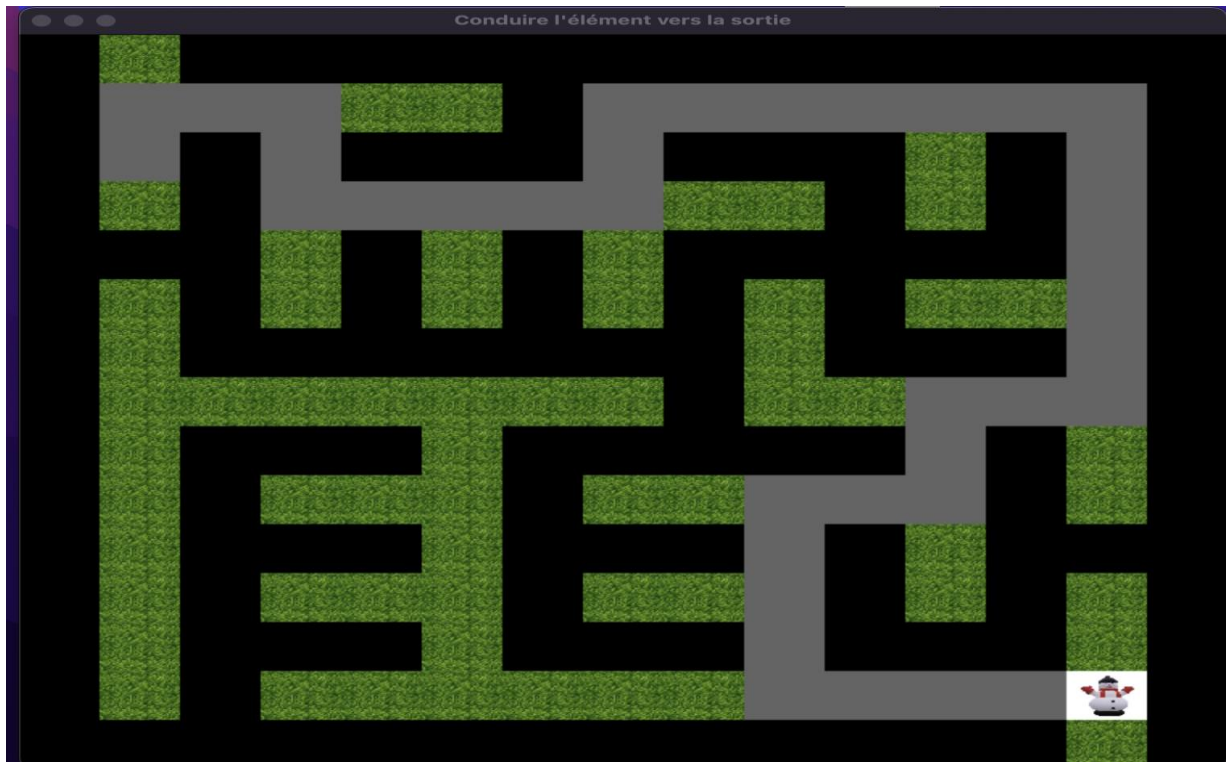
3. Avec le parcours en longueur

a. Principes

Le principe du DFS dans un labyrinthe consiste à explorer autant que possible dans une direction avant de revenir en arrière pour explorer d'autres directions. On utilise la récursivité ou une pile pour stocker les cases à explorer. À chaque étape, on explore une branche du labyrinthe aussi loin que possible, jusqu'à ce qu'on atteigne un mur ou la sortie. Si un mur est atteint, on revient en arrière pour explorer d'autres branches. Ce processus se poursuit jusqu'à ce que la sortie soit trouvée ou que toutes les branches possibles aient été explorées.

b. Démonstration





Conclusion

Dans ce projet, nous avons exploré en profondeur les notions fondamentales des graphes ainsi que les algorithmes essentiels pour les parcourir. Nous avons commencé par comprendre les différents types de graphes et leurs représentations, ce qui nous a permis de modéliser un labyrinthe sous forme de graphe implicite. Ensuite, nous avons étudié en détail les algorithmes de parcours en profondeur (DFS) et en largeur (BFS), ainsi que leur complexité associée. Nous avons également examiné comment ces algorithmes peuvent être appliqués dans le contexte spécifique du parcours d'un labyrinthe.

En choisissant Python comme langage de programmation, nous avons bénéficié de sa facilité d'utilisation et de ses bibliothèques riches en fonctionnalités pour manipuler des graphes. En particulier, nous avons discuté des principes du parcours en largeur dans un labyrinthe, mettant en lumière l'approche systématique pour explorer les cases voisines et calculer le chemin le plus court avec l'algorithme de Dijkstra.

Enfin, nous avons abordé les principes du parcours en longueur dans un labyrinthe, mettant en évidence la stratégie de exploration exhaustive pour découvrir tous les chemins possibles. Ce projet nous a permis de comprendre l'importance des graphes dans la résolution de problèmes informatiques et nous a fourni une base solide pour explorer davantage ce domaine fascinant.

➤ Références

Bibliographie

Introduction à l'algorithmique 2ème Edition, Thomas-H Cormen, Charles-E Leiserson et **Ronald-L Rivest, 2002.**

Sitographie

Algorithmes de graphe, [Cours_Algo_Graphes.pdf \(univ-paris13.fr\)](#)

Algorithmique et complexité, [Algorithmique et Complexité - Cours 1/7 : Parcours de graphes \(centralesupelec.fr\)](#)

Documentation Matplotlib, [Documentation de Matplotlib — Documentation de Matplotlib 3.8.3](#)

Documentation de PyGame, [Page d'accueil de Pygame — documentation de pygame v2.6.0](#)

Documents internes :

Cours Algorithmique et Complexité, Madame Kassé, 2024